Project - High Level Design

on

Multi-Agent

Insurance System

Course Name : Agentic AI

Institution Name: Medicaps University

–Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	S. JEZREEL	EN22IT301085
2	KRITAGYA TIWARI	EN22IT301048
3	SADHANA YADAV	EN24CA5030146
4	CHANDRESH BHATI	EN24CA5030040
5	SHIVKUMAR SOLANKI	EN24CA5030157

Group Name: 02D12

Project Number:AAI-40

Industry Mentor Name:Mr. Ashwin sir

University Mentor Name:Mr. Ajeet singh Rajput

Academic Year:2025-26

Table of Contents

1.Introduction
2.System Design.
2.1 Application Design
2.1.1 Class Diagram
2.1.2 Sequence Diagram
2.1.3 Activity Diagram
3. Data Design and Process flow
3.1. Data Model
3.2. Data Access Mechanism
3.3. Data Retention Policies
3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
7.1. Security Aspects
7.2. Performance Aspects
8. References

1. Introduction

The rapid development of Generative AI has enabled intelligent systems capable of performing complex tasks through collaboration between specialized agents. This project focuses on designing a **collaborative multi-agent system for insurance information generation**, where different AI agents work together to produce structured and meaningful outputs.

The system consists of two primary agents: a **Researcher Agent** responsible for gathering insurance-related information such as policy coverage, benefits, and conditions, and a **Writer Agent** responsible for organizing, synthesizing, and formatting this information into structured outputs. The agents communicate through defined protocols and share contextual information using state management techniques.

The system is implemented using **Python**, with frameworks such as langchain to orchestrate agent collaboration and workflow management. A **Large Language Model (LLM)** is used to process natural language queries and generate context-aware responses. The architecture demonstrates advanced AI design concepts such as **inter-agent communication, workflow orchestration, and modular agent specialization**.

1.1 Scope of the Document

This document provides a detailed **High Level Design (HLD)** of the collaborative insurance agent system. It describes the internal architecture components, pipelines and interactions between agents that enable the system to function effectively.

The document focuses on explaining how user queries are processed by specialized agents, how information flows between components, and how the final output is generated. It also provides details about system APIs, design considerations, and communication protocols used for agent collaboration. The purpose of this document is to offer a clear understanding of the system's implementation and technical structure.

1.2 Intended Audience

This document is intended for **software developers, AI engineers, project mentors, and system reviewers** who need to understand the internal design of the system. It provides detailed insights into the architecture, components, and workflow of the collaborative AI agent system.

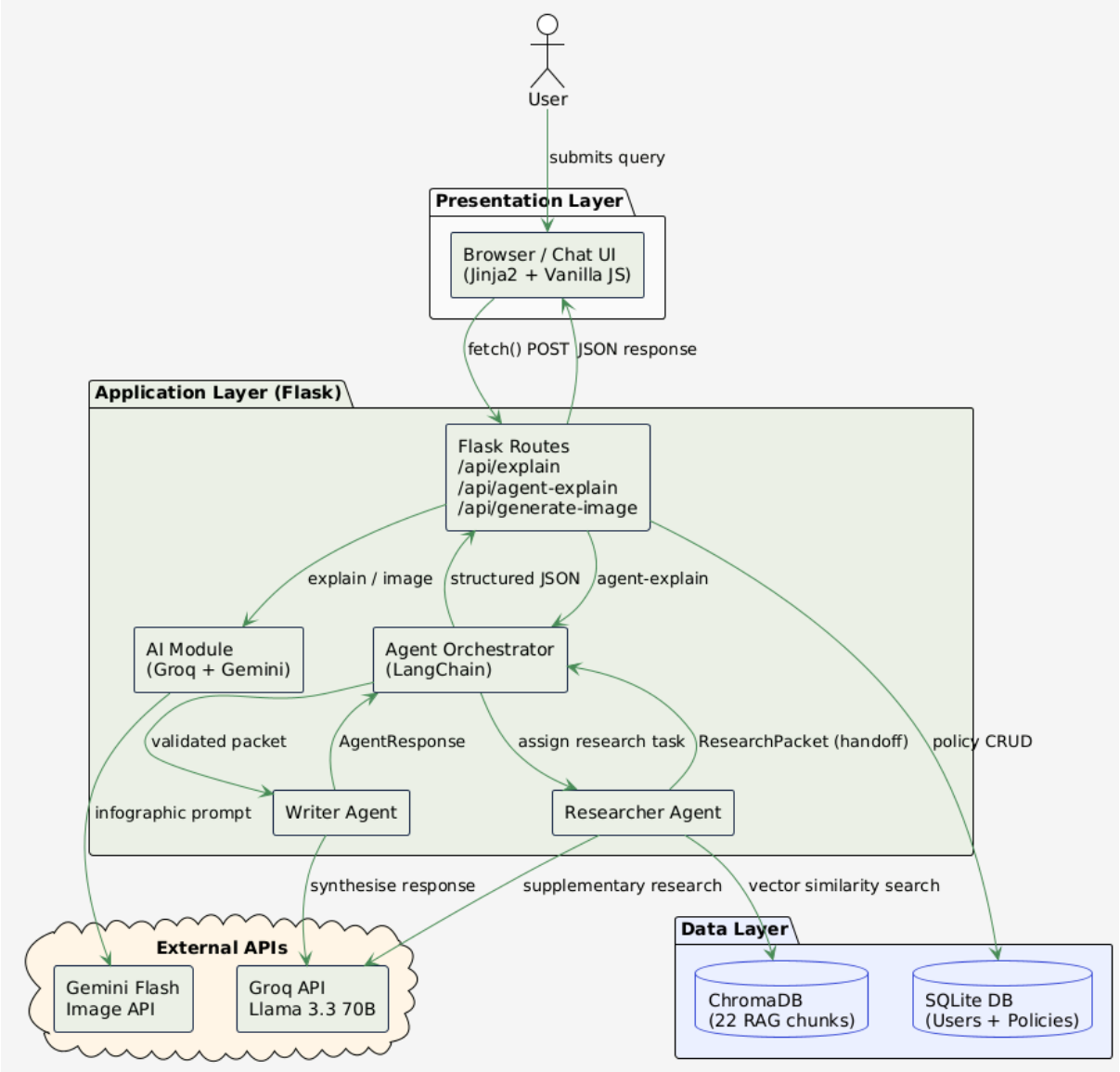
1.3 System Overview

Insurance Hub is a production-grade, cloud-deployed multi-agent AI platform that processes insurance-related queries through a collaborative pipeline of specialised AI agents. The system combines a Retrieval-Augmented Generation (RAG) knowledge base built on ChromaDB with two specialised LLM-powered agents – a Researcher Agent and a Writer Agent – orchestrated via LangChain to produce structured, India-specific insurance guidance for end users.

Beyond the agent pipeline, the application provides a complete insurance policy management system with full CRUD operations, user authentication, interactive policy information pages, and a multimodal AI Studio that generates both text explanations and visual infographics. The platform is containerised with Docker, orchestrated via Kubernetes (kubeadm) on AWS EC2, and exposes REST API endpoints consumed asynchronously by the frontend.

1.4 System Architecture Diagram :

The system architecture of the Collaborative Insurance Agents application is designed using a multi-agent approach, where different agents collaborate to process and generate structured insurance-related outputs. The system consists of a central Agent Orchestrator, implemented using LangChain which manages the workflow and communication between agents. ChromaDB vector database, which fetches from the data for the RAG agent. When a user submits a query, the orchestrator assigns the task to the Researcher Agent, which retrieves relevant insurance information using the LLM API. The collected data is then passed to the Writer Agent through a defined hand-off mechanism, where it is processed and formatted into a structured response using a predefined Prompt template.


User
submits query
Presentation Layer
Browser / Chat UI
(Jinja2 + Vanilla JS)
fetch() POST
JSON response
Application Layer (Flask)
Flask Routes
/api/explain
/api/agent-explain
/api/generate-image
explain / image
structured JSON
agent-explain
AI Module
(Groq + Gemini)
Agent Orchestrator
(LangChain)
validated packet
AgentResponse
assign research task
ResearchPacket (handoff)
policy CRUD
infographic prompt
Writer Agent
Researcher Agent
synthesise response
supplementary research
vector similarity search
External APIs
Gemini Flash
Image API
Groq API
Llama 3.3 70B
Data Layer
ChromaDB
(22 RAG chunks)
SQLite DB
(Users + Policies)

2. System Design

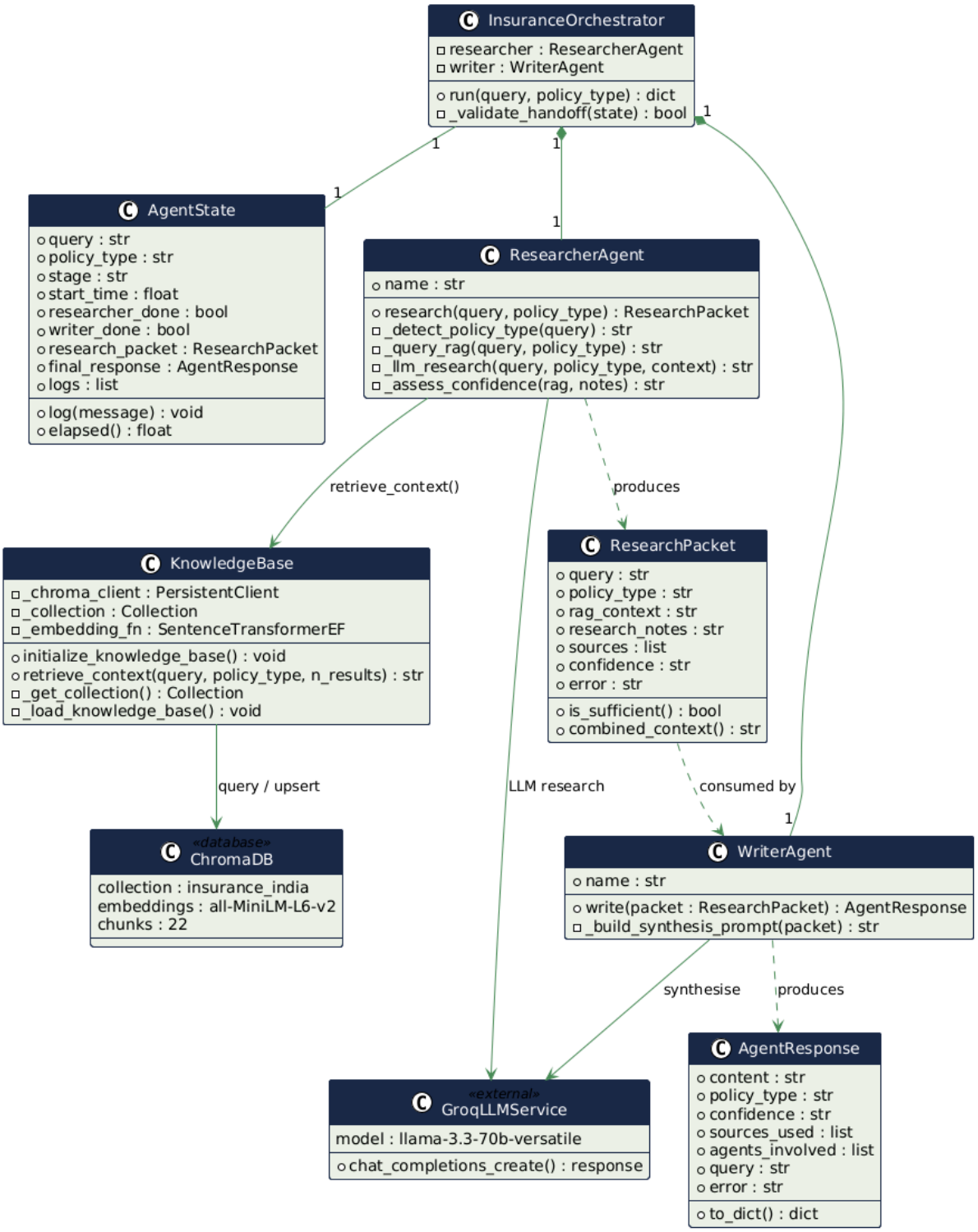
2.1 Application Design

Agent/ Component	Responsibility
Agent Orchestrator (LangChain)	Manages the complete pipeline lifecycle – assigns tasks, validates the handoff gate, maintains AgentState, and returns the final JSON response.
Researcher Agent	Detects policy type from the query, retrieves the top-4 relevant chunks from ChromaDB via cosine similarity, and runs supplementary LLM research (Groq, temperature 0.3) to fill gaps.
Handoff Gate	Validates the ResearchPacket produced by the Researcher Agent before forwarding it to the Writer Agent. Checks for non-empty context and logs confidence level (high / medium / low).
Writer Agent	Receives the validated ResearchPacket, synthesises RAG context and research notes, and produces a structured, Indian-market-focused response using Groq (temperature 0.5)

ChromaDB Vector Store	Stores 22 curated India-specific insurance knowledge chunks as vector embedding (sentence-transformers all-MiniLM-L6-v2). Serves similarity-ranked context to the Researcher Agent.
LLM Service (Groq)	Provides sub-2-second LLM inference for both agents using Llama 3.3 70B Versatile.

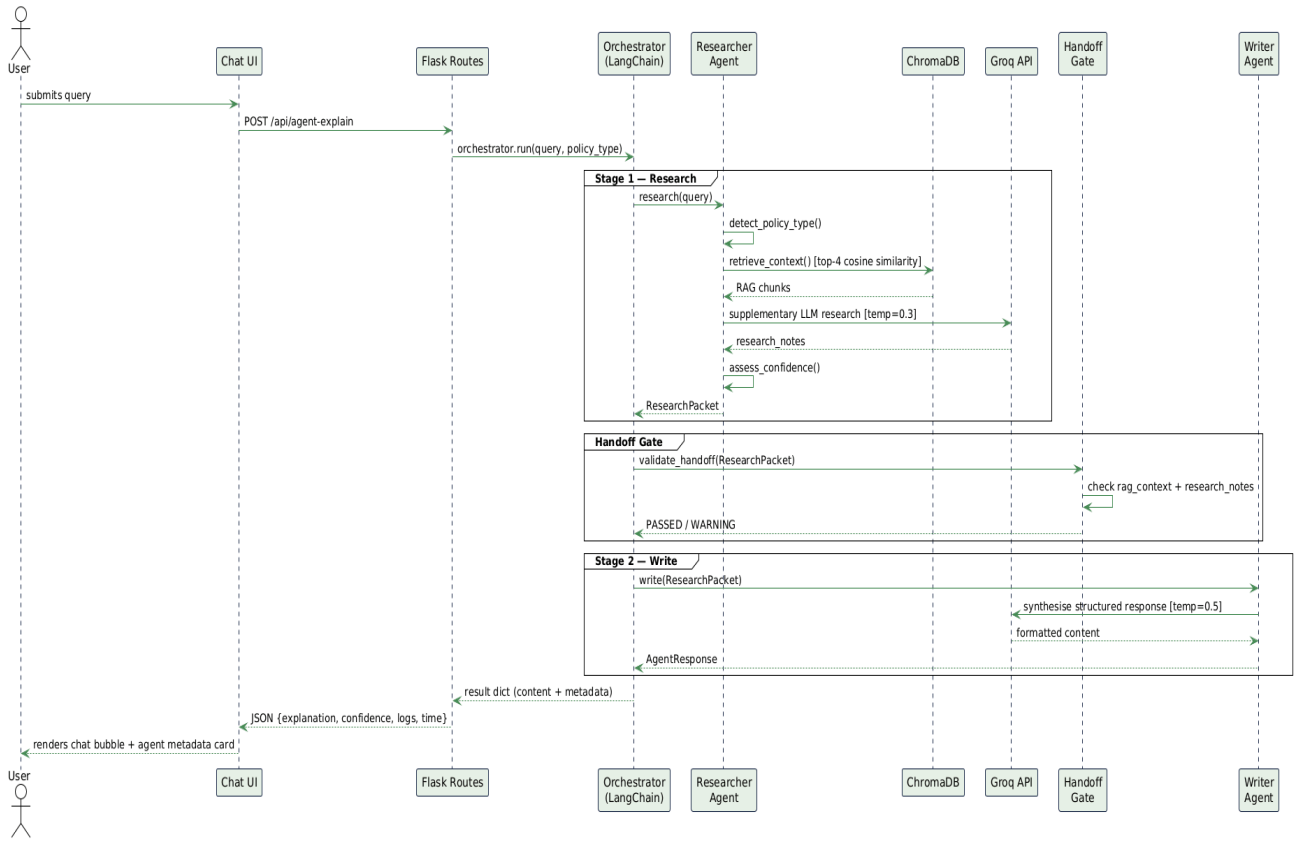
2.1.1 Class Diagram

The class diagram represents the structural design of the collaborative multi-agent system. It defines key classes such as User, Agent Orchestrator, Researcher Agent, Writer Agent, and LLM Service. The Agent Orchestrator acts as the central controller that manages task distribution and workflow execution. The Researcher Agent is responsible for analyzing user queries and retrieving relevant insurance data, while the Writer Agent formats and generates structured outputs. The LLM Service provides natural language processing capabilities used by both agents. This design ensures modularity, reusability, and clear separation of responsibilities, making the system scalable and easy to maintain.

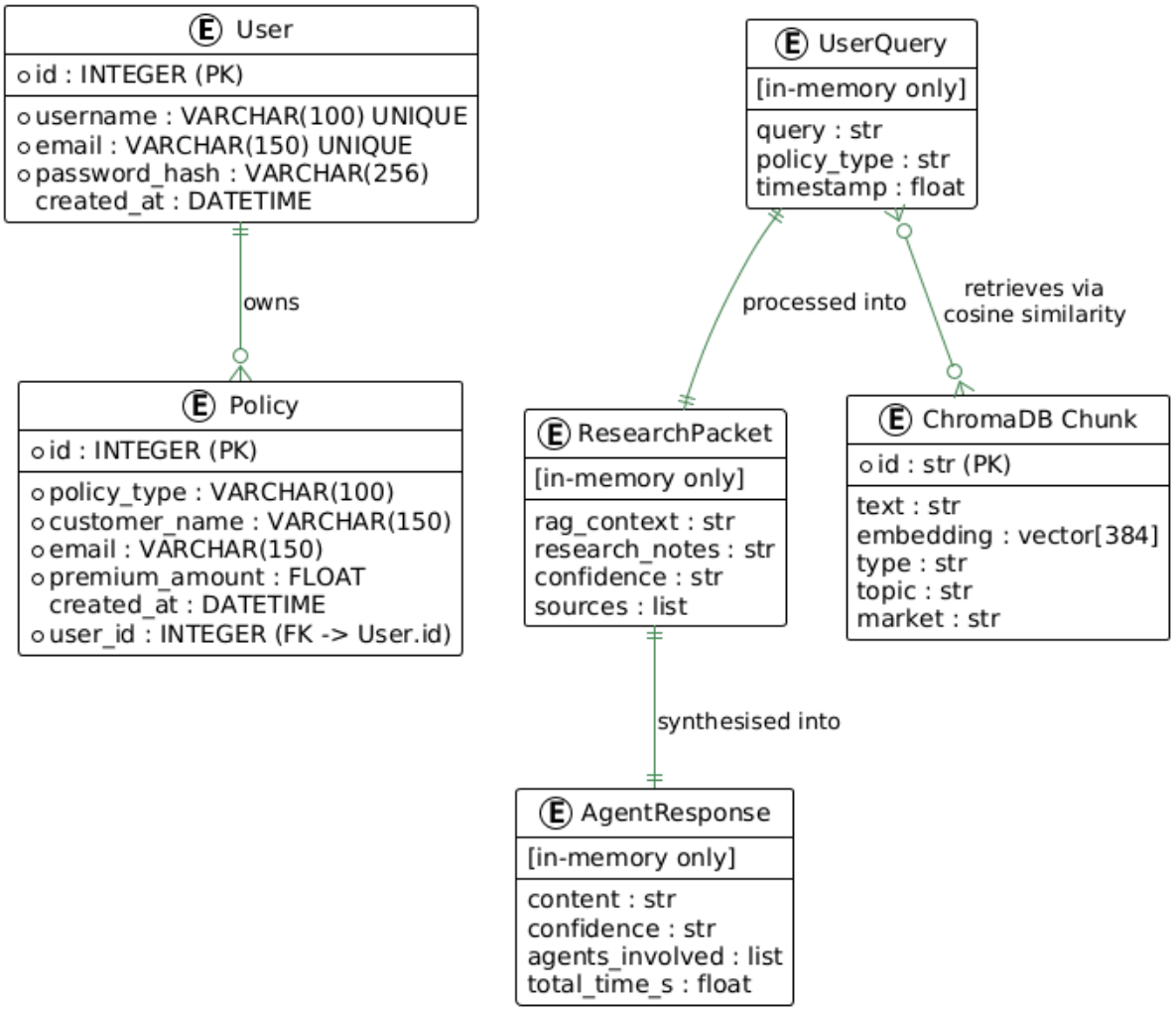

InsuranceOrchestrator
researcher : ResearcherAgent
writer : WriterAgent
run(query, policy_type) : dict
_validate_handoff(state) : bool
AgentState
query : str
policy_type : str
stage : str
start_time : float
researcher_done : bool
writer_done : bool
research_packet : ResearchPacket
final_response : AgentResponse
logs : list
log(message) : void
elapsed() : float
ResearcherAgent
name : str
research(query, policy_type) : ResearchPacket
_detect_policy_type(query) : str
_query_rag(query, policy_type) : str
_llm_research(query, policy_type, context) : str
_assess_confidence(rag, notes) : str
retrieve_context()
produces
KnowledgeBase
_chroma_client : PersistentClient
_collection : Collection
_embedding_fn : SentenceTransformerEF
initialize_knowledge_base() : void
retrieve_context(query, policy_type, n_results) : str
_get_collection() : Collection
_load_knowledge_base() : void
ResearchPacket
query : str
policy_type : str
rag_context : str
research_notes : str
sources : list
confidence : str
error : str
is_sufficient() : bool
combined_context() : str
query / upsert
LLM research
consumed by
«database»
ChromaDB
collection : insurance_india
embeddings : all-MiniLM-L6-v2
chunks : 22
WriterAgent
name : str
write(packet : ResearchPacket) : AgentResponse
_build_synthesis_prompt(packet) : str
synthesise
produces
«external»
GroqLLMService
model : llama-3.3-70b-versatile
chat_completions_create() : response
AgentResponse
content : str
policy_type : str
confidence : str
sources_used : list
agents_involved : list
query : str
error : str
to_dict() : dict

2.1.2 Sequence Diagram

The sequence diagram illustrates the dynamic interaction between system components over time. When a user submits a query, the Agent Orchestrator receives and processes it. The Researcher Agent is first invoked to retrieve relevant insurance information using the LLM API. The intermediate data is then passed to the Writer Agent through the orchestrator. The Writer Agent formats the content into structured output using the LLM API. Finally, the processed response is returned to the user. This diagram highlights inter-agent communication, task sequencing, and dependency on the LLM service.



2.1.3 ER Diagram



3.1 Process Flow

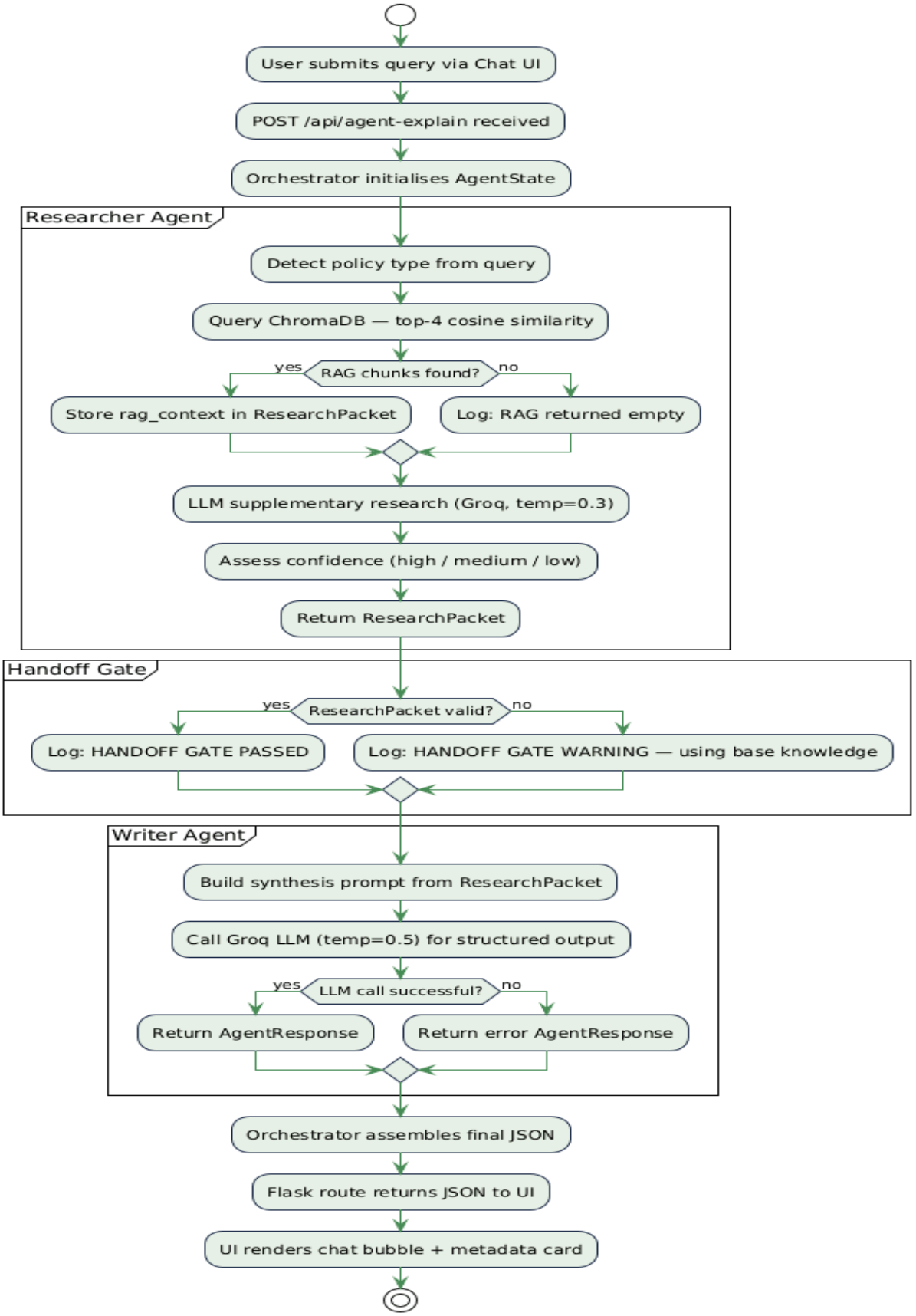
The process begins when the user submits an insurance-related query to the system. The query is sent to the **Researcher Agent**, which analyzes the request and retrieves relevant information using the LLM API.

Once the required data is gathered, the Researcher Agent sends the intermediate output to the **Writer Agent** through a defined hand-off protocol. The Writer Agent processes the received information and generates a structured response, including organized explanations or summaries. Finally, the system returns the formatted output to the user.

3.2 Information Flow

The information flow within the system follows a sequential pipeline. User input is first captured by the application interface and forwarded to the Researcher Agent. The Researcher Agent processes the query and generates structured intermediate information.

This intermediate data is passed to the Writer Agent, which refines and organizes the information into a clear and structured output. The final output is then delivered back to the user. The system maintains context between agents using shared state management techniques to ensure consistency and continuity.


User submits query via Chat UI
POST /api/agent-explain received
Orchestrator initialises AgentState
Researcher Agent
Detect policy type from query
Query ChromaDB — top-4 cosine similarity
yes
RAG chunks found?
no
Store rag_context in ResearchPacket
Log: RAG returned empty
LLM supplementary research (Groq, temp=0.3)
Assess confidence (high / medium / low)
Return ResearchPacket
Handoff Gate
yes
ResearchPacket valid?
no
Log: HANDOFF GATE PASSED
Log: HANDOFF GATE WARNING — using base knowledge
Writer Agent
Build synthesis prompt from ResearchPacket
Call Groq LLM (temp=0.5) for structured output
yes
LLM call successful?
no
Return AgentResponse
Return error AgentResponse
Orchestrator assembles final JSON
Flask route returns JSON to UI
UI renders chat bubble + metadata card

3.3 Components Design

The system is composed of several key components that work together to implement the collaborative workflow.

The User Interface component allows users to submit queries and view results. The **Agent Orchestrator** coordinates the execution of tasks and manages communication between agents. The **Researcher Agent** retrieves and analyzes relevant insurance data. The **Writer Agent** converts the information into structured and readable content.

Additionally, the LLM API component provides natural language understanding and generation capabilities, enabling the system to process complex queries and produce meaningful responses.

3.4 Key Design Considerations

Several important design considerations were taken into account while developing the system.

First, **modularity** ensures that each agent performs a specific task independently. This simplifies development and allows future extensions. Second, efficient communication protocols are used to enable smooth interaction between agents. Third, state management mechanisms maintain context across different stages of the workflow. The system also focuses on scalability and flexibility, allowing additional agents to be introduced in the future for tasks such as validation, summarization, or visualization.

3.5 API Catalogue

The Multi Agent Insurance System integrates external LLM APIs and internal agent-based APIs to perform its core functionalities. The external APIs provide natural language processing capabilities, while the internal APIs manage agent communication, task orchestration, and workflow execution between the Researcher and Writer agents.

3.5.1 Internal REST API endpoints

Endpoint [Method]	Description
POST /api/agent-explain	Multi-agent RAG pipeline. Returns structured explanation + pipeline metadata (confidence, logs, agents involved, time taken).
POST /api/explain	Single-LLM fast explanation (no RAG). Used by AI Studio Policy Explainer tab.
POST /api/generate-image	Text-to-image infographic. Tries Gemini Flash → HF FLUX.1-Dev. Returns base64 data URL.
GET /ai-studio	Renders the AI Studio chat UI (Policy Explainer + Infographic Generator tabs).
GET /agent-ui	Renders the Multi-Agent RAG Pipeline chat UI with pipeline step visualisation.
GET/POST /add-policy	Create a new insurance policy record for the logged-in user.

GET/POST /edit/<id>	Update an existing policy. Validates ownership before allowing changes.
POST /delete/<id>	Delete a policy. Validates ownership before deletion.
GET /	Dashboard — displays all policies owned by the current user.
GET/POST /register	User registration with password hashing (Werkzeug).
GET/POST /login	User login with session management (Flask-Login).
GET /logout	Clears user session and redirects to login.

3.5.2 External REST API endpoints

Service	Details
Groq API — Llama 3.3 70B	Used by both agents. Researcher: temperature 0.3 (factual). Writer: temperature 0.5 (structured). Max tokens: 800 (researcher) / 900 (writer).
Gemini Flash Image API	Primary image generator. Model: gemini-3.1-flash-image-preview. 500 free requests/day. responseModalities: [TEXT, IMAGE].
Hugging Face Inference	Fallback image generator. Model: FLUX.1-Dev (same model as Raphael AI). provider=auto selects fastest available.
sentence-transformers	Local embedding model (all-MiniLM-L6-v2) used by ChromaDB for chunk embedding — no external API call needed.

3.6 Non Functional requirements

3.6.1 API Interaction Flow

The API interaction begins when the user submits a query through the system interface. The query is sent to the internal API, which triggers the Agent Orchestrator. The orchestrator forwards the request to the Researcher Agent, which uses the LLM API to retrieve relevant insurance information. The intermediate data is then passed through the Agent Communication API to the Writer Agent. The Writer Agent processes the data using the LLM API and generates a structured response. Finally, the system returns the formatted output to the user.

3.6.2 Data Design

The data design of the Multi Agent Insurance System focuses on efficient handling and flow of information between system components. Instead of relying on heavy database storage, the system uses a dynamic, in-memory data processing approach. User queries are processed by the Researcher Agent to generate intermediate data, which is then passed to the Writer Agent for structured output generation. The Agent Orchestrator ensures smooth coordination and data transfer between agents. This design promotes modularity, scalability, and faster processing. By minimizing persistent storage and emphasizing real-time data flow, the system achieves high performance while maintaining flexibility and simplicity.

3.6.3 Data Model

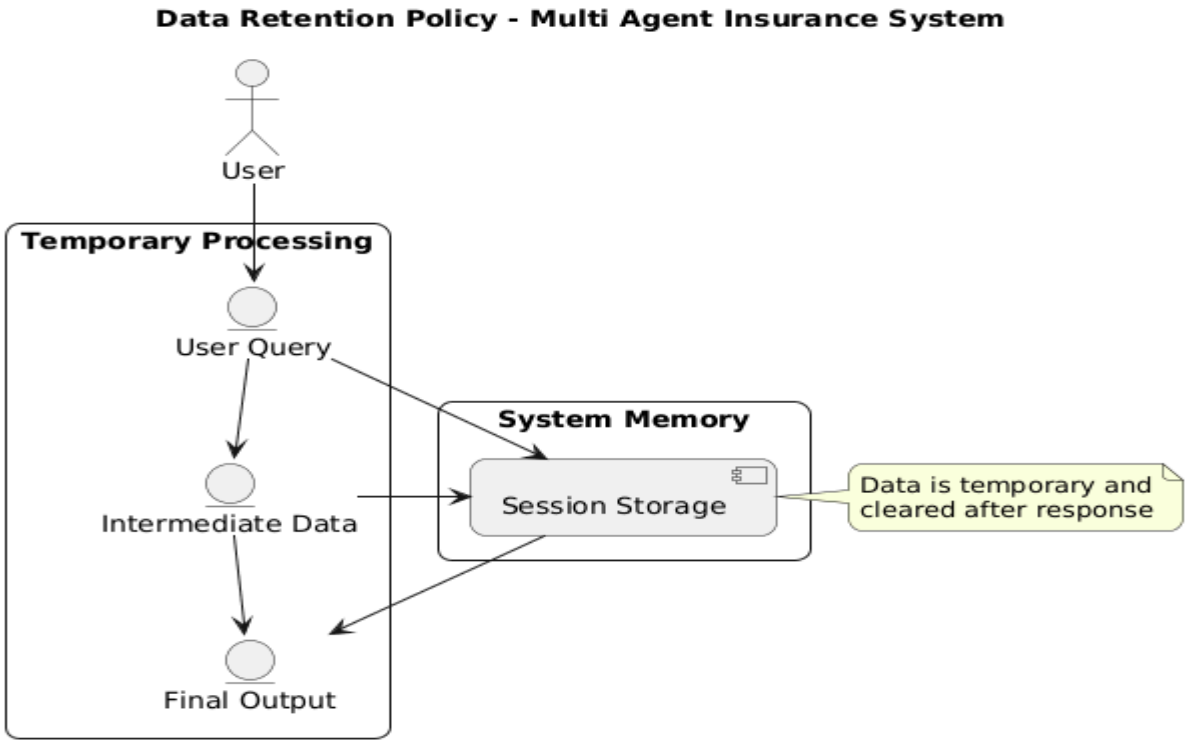
The data model of the Multi Agent Insurance System represents the logical structure and flow of information across components. It consists of three primary entities: User Query, Intermediate Data, and Final Output. The User Query captures the input provided by the user. The Researcher Agent processes this input and generates Intermediate Data, which contains extracted insurance-related insights. This data is then forwarded to the Writer Agent, which produces the Final Output in a structured and user-friendly format. The model ensures a clear separation between raw and processed data, enabling efficient collaboration between agents and supporting maintainability and scalability of the system.

3.6.4 Data Access Mechanism

The system uses an agent-driven data access mechanism rather than traditional database queries. The Researcher Agent retrieves relevant information dynamically through interaction with the LLM API, based on the user query. The Agent Orchestrator manages the flow of this data and ensures it is correctly transferred to the Writer Agent. The Writer Agent accesses the intermediate data through a shared state and processes it further to generate structured outputs. This approach enables real-time data processing, reduces dependency on persistent storage, and enhances system flexibility. It also ensures efficient communication between agents within the multi-agent architecture.

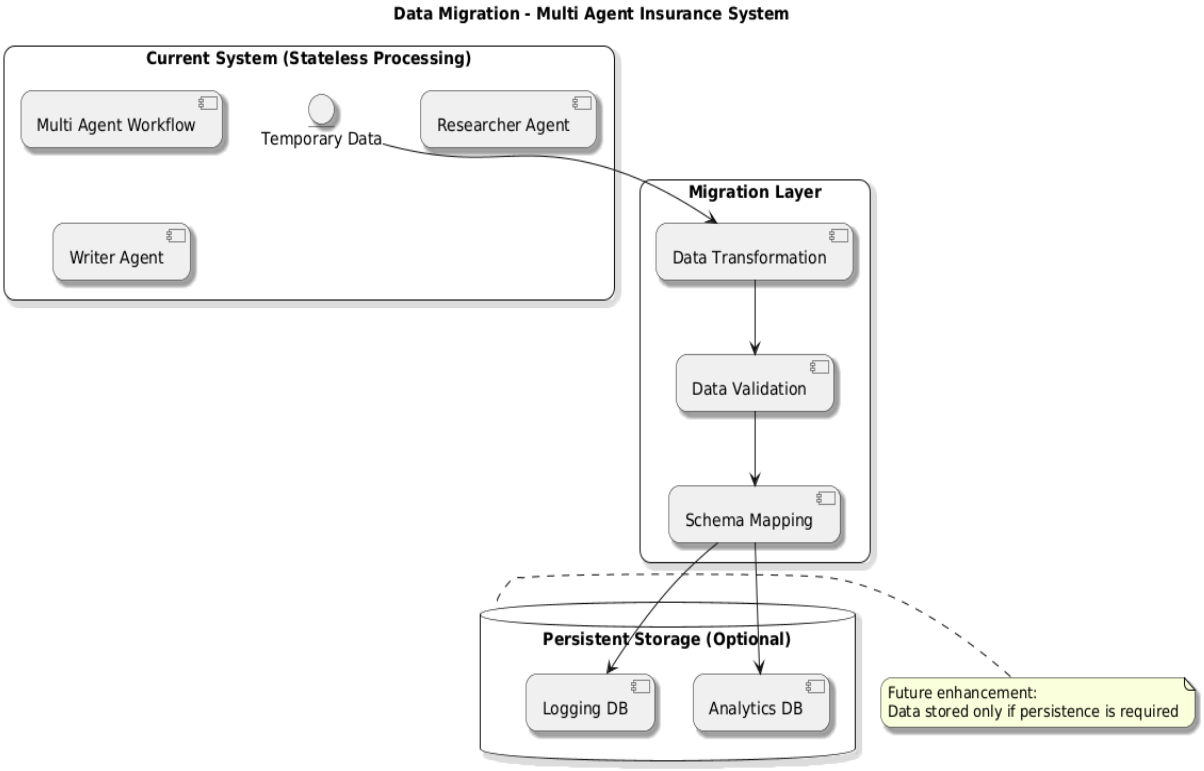
3.6.5 Data Retention Policies

The Multi Agent Insurance System follows a minimal data retention policy to ensure privacy and efficiency. User queries, intermediate data, and generated outputs are processed only during the execution of a request and are not stored permanently. Temporary data is maintained in session memory for coordination between agents and is cleared after the response is generated. This approach reduces storage overhead and minimizes security risks associated with sensitive data. By avoiding long-term storage of user information, the system aligns with best practices for data privacy and secure system design while maintaining efficient processing of insurance-related queries.



3.6.6 Data Migration

Data migration requirements in the Multi Agent Insurance System are minimal due to its dynamic and stateless data processing approach. Since the system does not depend heavily on persistent databases, there is limited need for transferring or transforming stored data. However, if future enhancements introduce persistent storage for analytics or logging purposes, data migration can be implemented using standard techniques such as data transformation, validation, and schema mapping. The modular architecture ensures that migration processes can be integrated without disrupting the agent workflow. This provides flexibility for future scalability while maintaining the simplicity of the current system design.


Data Migration - Multi Agent Insurance System
Current System (Stateless Processing)
Multi Agent Workflow
Temporary Data
Researcher Agent
Writer Agent
Migration Layer
Data Transformation
Data Validation
Schema Mapping
Persistent Storage (Optional)
Logging DB
Analytics DB
Future enhancement:
Data stored only if persistence is required

4. Interfaces

The Multi Agent Insurance System provides multiple interfaces to enable interaction between users, internal agents, and external AI services. It includes user interfaces for query input, internal interfaces for agent communication, and external APIs for language processing. These interfaces ensure seamless coordination between the Agent Orchestrator, Researcher Agent, and Writer Agent. The design supports efficient data exchange and structured workflow execution, enabling the system to generate high-quality insurance-related responses through collaborative agent processing.

4.1 User Interface

The user interface provides a simple and interactive platform for users to submit insurance-related queries and receive structured responses. It is designed to be responsive and user-friendly, allowing easy interaction with the system. The interface includes input fields for queries and output sections for displaying generated results. It ensures smooth navigation and real-time interaction, enabling users to easily access the system's functionality without complexity.

4.2 Application Interface

The application interface is responsible for handling internal system operations and managing communication between components. It acts as a bridge between the user interface and the multi-agent system. The Agent Orchestrator processes incoming requests, coordinates tasks between the Researcher Agent and Writer Agent, and manages the overall workflow. This structured interaction ensures modularity, efficient processing, and clear separation of responsibilities within the system.

4.3 External API Interface

The system integrates external LLM APIs to enable natural language understanding and generation. The Researcher Agent uses the API to retrieve relevant insurance information, while the Writer Agent uses it to generate structured outputs. These APIs are accessed securely using tokens stored in environment variables. The integration of external AI services enhances the system's ability to process complex queries and generate meaningful responses.

4.4 Data Exchange Format

Data exchange within the system is carried out through structured communication between components. User inputs are processed as text queries and passed through the Agent Orchestrator. Intermediate data generated by the Researcher Agent is transferred to the Writer Agent in a structured format. The final output is generated as formatted text and presented to the user. This standardized data flow ensures consistency, reduces complexity, and supports efficient communication across the system.

4.5 Error Handling Interface

The system includes error handling mechanisms to manage invalid inputs, processing issues, and API failures. Input validation ensures that only valid queries are processed. The system handles exceptions gracefully and provides appropriate responses to users in case of errors. This improves system reliability and ensures a smooth user experience by clearly communicating issues and maintaining stable system behavior.

5. State and Session Management

The Multi Agent Insurance System uses a lightweight and efficient state management approach to handle data flow during the execution of user requests. Since the system is primarily designed as a stateless architecture, it does not rely on long-term session storage. Instead, it maintains temporary state information during the lifecycle of a single request.

When a user submits a query, the Agent Orchestrator manages the workflow and maintains the context of the request. Intermediate data generated by the Researcher Agent is stored temporarily in shared memory or a state manager, allowing the Writer Agent to access and process it. This ensures smooth coordination and accurate transformation of data across agents.

Session management is handled at a request level, where each user interaction is treated independently. No persistent session data is stored beyond the execution cycle, ensuring data privacy and reducing system complexity. This design prevents data overlap between users and ensures scalability by allowing multiple requests to be processed concurrently without interference.

Overall, the combination of temporary state handling and stateless session design enhances system performance, simplifies architecture, and ensures secure and efficient processing of insurance-related queries.

6. Caching

Caching in the Multi Agent Insurance System is used to improve performance and reduce repeated calls to the Groq API. Since the system relies on real-time LLM processing for both the Researcher Agent and Writer Agent, caching can store frequently requested queries and their generated responses. This reduces latency and minimizes API usage cost. While the current system primarily processes queries dynamically, integrating caching at different levels can significantly enhance efficiency, scalability, and overall system performance in high-load scenarios.

6.1 Static Content Caching

Static resources such as UI components, scripts, and styles can be cached at the client side to improve load time and responsiveness. This reduces repeated requests to the server and ensures smooth interaction with the system interface.

6.2 Server-Side Caching

Server-side caching can be implemented to store responses generated by the multi-agent workflow. The Agent Orchestrator can first check cached outputs before sending requests to the Groq API. This reduces redundant execution of Researcher and Writer agents and optimizes system performance.

6.3 API Response Handling

The system currently generates responses dynamically using the Groq API to maintain accuracy and relevance. However, repeated or similar queries can benefit from caching previously generated outputs. This helps reduce API calls, lower response time, and improve system efficiency without compromising output quality.

6.4 Cache Invalidation

Cache invalidation ensures that outdated or irrelevant responses are removed or refreshed. Time-based expiration or query-based validation can be used to maintain accuracy. This ensures that users always receive up-to-date and relevant insurance information.

7. Non-Functional Requirements

Requirement	Implementation
Performance	Groq API provides sub-2s LLM responses. Async JS fetch() prevents UI blocking during 10-60s agent pipeline execution.
Scalability	Stateless per-request agent pipeline supports concurrent users. Kubernetes deployment with configurable replicas.
Security	API keys stored in .env (never in source). Flask-Login session management. Policy ownership enforced on all mutations. Password hashing via Werkzeug PBKDF2-SHA256.
Reliability	Multi-provider image generation fallback (Gemini → HF FLUX). Agent pipeline degrades gracefully if RAG unavailable.
Modularity	Each agent is an independent class with a single responsibility. New agents can be added without modifying existing ones.

Privacy	Agent state is fully ephemeral — cleared after each request. No query or intermediate data stored permanently.
Deployability	Dockerfile + Kubernetes manifests (deployment.yaml + service.yaml). NodePort 30007 on AWS EC2. GitHub repository for version control.

7.1 Security Aspects

API keys (Groq, Gemini, HuggingFace) stored exclusively in .env – never in source code or templates All routes protected with @login_required decorator; unauthenticated requests redirected to login. Policy ownership enforced on every edit/delete operation – cross-user data access is prevented at the route level. Passwords hashed with Werkzeug generate_password_hash (PBKDF2-SHA256 algorithm). Flask SECRET_KEY used for cryptographically signed session cookies Database transactions wrapped in try/except with rollback on failure – prevents partial/corrupt write. Agent state is fully ephemeral – user queries and intermediate data are never persisted. Input validation on all form fields and API endpoints before processing.

7.2 Performance Aspects

Performance in the Multi Agent Insurance System focuses on fast response generation, efficient agent coordination, and optimized use of external APIs. The system ensures real-time processing of queries while maintaining high-quality outputs. It is designed to handle multiple simultaneous requests efficiently, ensuring stable performance even under increased load. Proper resource utilization and optimized workflows contribute to consistent and reliable system behavior.

7.2.1 Agent Workflow Efficiency

The Agent Orchestrator manages task execution between the Researcher and Writer agents, ensuring smooth data flow and minimal processing delays. This structured workflow reduces redundancy and improves overall system efficiency.

7.2.2 LLM API Optimization (Groq API Usage)

The system optimizes API usage by structuring prompts efficiently and reducing unnecessary calls to the Groq API. This improves response time, reduces latency, and ensures effective utilization of external AI services.

7.2.3 Scalability of Multi-Agent System

The system is designed to handle multiple user requests simultaneously. Its modular architecture allows easy scaling by adding more agents or enhancing processing capabilities, ensuring consistent performance under high demand.

7.2.4 Future Performance Improvements

Future enhancements may include caching, asynchronous processing, and load balancing to further reduce latency and improve efficiency. These improvements will enable the system to handle larger workloads and provide faster responses.

References

1. Python (Programming Language)

Use: Used as the core programming language to implement the multi-agent system, handle orchestration logic, and integrate LLM-based processing.
Link:
https://docs.python.org/3/

2. LangChain

Use: Used for building and managing agent-based workflows, enabling communication between Researcher and Writer agents, and handling prompt chaining.
Link:
https://docs.langchain.com/

3. CrewAI

Use: Used to design and manage collaborative multi-agent systems with defined roles, responsibilities, and hand-off mechanisms between agents.
Link:
https://docs.crewai.com/

4. Groq API

Use: Used for LLM-based text generation. It powers both the Researcher Agent (information retrieval) and Writer Agent (structured output generation).
Link:
https://console.groq.com/docs

6. Docker

Use: Used to containerize the application and ensure consistent deployment across different environments such as local systems

and cloud platforms.

Link:
https://docs.docker.com/

7. Kubernetes

Use: Used for managing, scaling, and orchestrating containerized applications in production environments.
Link:
https://kubernetes.io/docs/